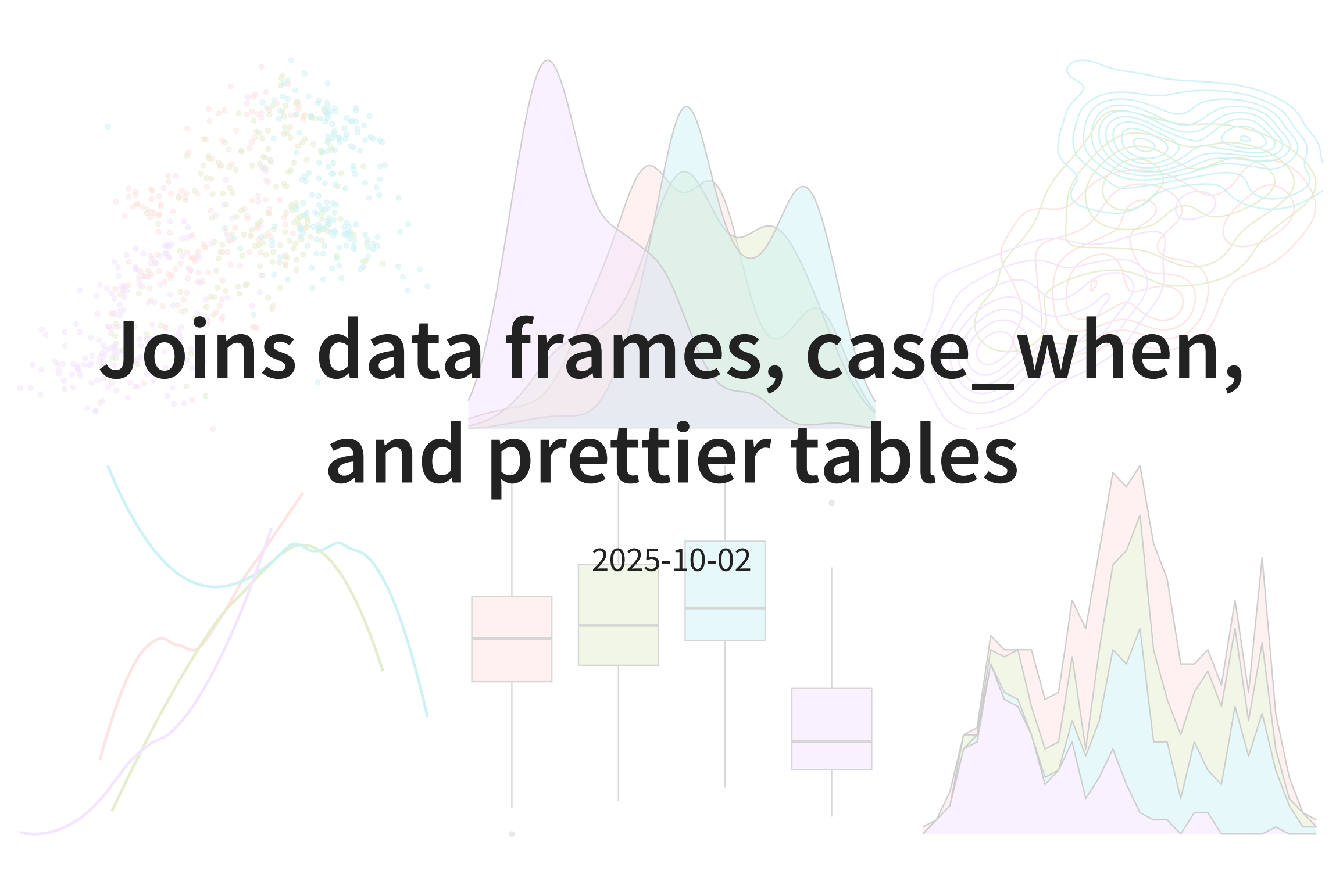


The background of the slide is a collage of various data visualization plots. In the top left, there is a scatter plot with points in purple, pink, green, and blue. To its right is a density plot with four overlapping bell-shaped curves in purple, pink, green, and blue. In the top right corner is a contour plot with concentric lines in blue, green, and red. The bottom left features a line plot with four colored lines (purple, green, red, blue) showing different trends. In the bottom center, there are four box plots in pink, green, blue, and purple, with the date '2025-10-02' written above the blue one. Finally, the bottom right shows an area chart with four overlapping colored areas (purple, green, blue, red) representing cumulative data over time.

Joins data frames, case_when, and prettier tables

2025-10-02

Housekeeping

Content through today is fair game for midterm!

Joining data frames

Assume we have two data frames, `df1` and `df2`. There are some shared variable(s) across the two.

We want to combine `df1` and `df2` into one single data frame. The `dplyr` code will look something like:

```
1 xxx_join(df1, df2)
```

- How do we combine `df1` and `df2` them? The following mutating joins give us rules:
 - `left_join()`: keep all cases from `df1`
 - `right_join()`: keep all cases from `df2`
 - `full_join()`: keep all cases from both `df1` and `df2`
 - `inner_join()`: keep all cases from `df1` where there are matching values in `df2` (more complicated in the scenario of multiple matches)

I know this is confusing! Examples will help clarify!

Setup

For the next few slides...

1 df1

ID	name
1	x1
2	x2
3	x3

1 df2

ID	grade
1	y1
2	y2
4	y4

I recommend writing these two down, as we will work with them in the next few slides.

Note! The variable **ID is shared between these two data frames. We will “join on” **ID**.**

left_join(df1, df2)

Adds columns to **df1** from **df2**, matching the cases in **df1**. Retains ALL cases from left data frame **df1**, and adds **NA** for variables from **df2** that have no match.

```
1 df1
```

ID	name
1	x1
2	x2
3	x3

```
1 df2
```

ID	grade
1	y1
2	y2
4	y4

```
1 left_join(df1, df2, by = "ID")
```

ID	name	grade
1	x1	y1
2	x2	y2
3	x3	NA

left_join(x, y)



Think of left data frame as fixed.

right_join(df1, df2)

Adds columns to `df1` from `df2`, matching the cases in `df2`. Retains ALL cases from right data frame `df2`, and adds `NA` for variables from `df1` that have no match.

```
1 df1
```

ID	name
1	x1
2	x2
3	x3

```
1 df2
```

ID	grade
1	y1
2	y2
4	y4

```
1 right_join(df1, df2, by = "ID")
```

ID	name	grade
1	x1	y1
2	x2	y2
4	NA	y4

right_join(x, y)

1	x1
2	x2
3	x3

1	y1
2	y2
4	y4

Think of right data frame as fixed

full_join(df1, df2)

Adds columns to **df1** from **df2**, *fully* combining the two data frames, matching when possible. Retains ALL cases from **df1** AND **df2**, and adds **NAs** as necessary.

```
1 df1
```

ID	name
1	x1
2	x2
3	x3

```
1 df2
```

ID	grade
1	y1
2	y2
4	y4

```
1 full_join(df1, df2, by = "ID")
```

ID	name	grade
1	x1	y1
2	x2	y2
3	x3	NA
4	NA	y4

full_join(x, y)



No chance for losing information.

inner_join(df1, df2)

Adds columns to `df1` from `df2`, retaining only cases that exist in both `df1` AND `df2`.

```
1 df1
```

ID	name
1	x1
2	x2
3	x3

```
1 df2
```

ID	grade
1	y1
2	y2
4	y4

```
1 inner_join(df1, df2, by = "ID")
```

ID	name	grade
1	x1	y1
2	x2	y2

inner_join(x, y)

1	x1	1	y1
2	x2	2	y2
3	x3	4	y4

Comprehension check

1. In which of the joins are **NAs** possibly introduced? In which of the joins are **NAs** never going to be introduced?
2. If we have some **df1** and **df2**, in what scenario will the result of the four joins be the same? (i.e. what has to be true about **df1** and **df2**?)
3. What do you think we call these “mutating” joins? *Think about where we’ve seen that verb before!*

- Note: it’s possible to join on more than one variable!

Example

We have data on fishery harvests (in tons) by countries from 2016, stored in data frame `fish`:

What does this code do?

```
1 fish |>
2   slice(1:8)
```

country	capture	aquaculture
Afghanistan	1000	1200
Albania	7886	950
Algeria	95000	1361
American Samoa	3047	20
Andorra	0	0
Angola	486490	655
Antigua and Barbuda	3000	10
Argentina	755226	3673

Suppose I would like to explore the data on a continent level. We have another data frame called `continent` that looks like this:

```
1 continents |>
2   slice(1:8)
```

Country	continent
Afghanistan	Asia
Åland Islands	Europe
Albania	Europe
Algeria	Africa
American Samoa	Oceania
Andorra	Europe
Angola	Africa
Anguilla	Americas

Discuss

`fish` data frame snapshot:

country	capture	aquaculture
Afghanistan	1000	1200
Albania	7886	950
Algeria	95000	1361
American Samoa	3047	20
Andorra	0	0

`continents` data frame snapshot:

Country	continent
Afghanistan	Asia
Åland Islands	Europe
Albania	Europe
Algeria	Africa
American Samoa	Oceania

- We want to keep all rows and columns from `fish` and add a column for corresponding continents. Which join function should we use?
- We want to keep only the rows from `fish` for which we have a corresponding continent and add a column for corresponding continents. Which join function should we use?
- Which variable(s) are we joining on?

Example (cont.)

Question 1:

```
1 left_join(fish, continents,  
2           by = c("country" = "Country")) |>  
3 slice(1:8)
```

country	capture	aquaculture	continent
Afghanistan	1000	1200	Asia
Albania	7886	950	Europe
Algeria	95000	1361	Africa
American Samoa	3047	20	Oceania
Andorra	0	0	Europe
Angola	486490	655	Africa
Antigua and Barbuda	3000	10	NA
Argentina	755226	3673	Americas

- Notice the **NA**
- Notice we don't have two copies of same **country/Country** variable!

Question 2:

```
1 inner_join(fish, continents,  
2            by = c("country" = "Country")) |>  
3 slice(1:8)
```

country	capture	aquaculture	continent
Afghanistan	1000	1200	Asia
Albania	7886	950	Europe
Algeria	95000	1361	Africa
American Samoa	3047	20	Oceania
Andorra	0	0	Europe
Angola	486490	655	Africa
Argentina	755226	3673	Americas
Armenia	3758	16381	Asia

Piping into `xxx_join()`

In the following line of code, when would an NA be introduced? (Answer forthcoming.)

```
1 right_join(fish, continents, by = c("country" = "Country"))
```

- Remember in data wrangling, we pipe from a previous line of code directly into another
- We can pipe directly into the join functions! The following achieves the same as above:

```
1 fish |>
2   right_join(continents, by = c("country" = "Country"))
```

Answer to previous question

NAs would be introduced if there is a country in `continents` that does not appear in `fish`. See the last 50-ish cases:

Show

10

 entries

Search:

	country	capture	aquaculture	continent
1	Afghanistan	1000	1200	Asia
2	Albania	7886	950	Europe
3	Algeria	95000	1361	Africa
4	American Samoa	3047	20	Oceania
5	Andorra	0	0	Europe
6	Angola	486490	655	Africa
7	Argentina	755226	3673	Americas
8	Armenia	3758	16381	Asia

	country	capture	aquaculture	continent
9	Aruba	142	0	Americas
10	Australia	174629	96847	Oceania

Showing 1 to 10 of 246 entries

Live code

- Using wrangling to obtain probabilities
- `case_when()` to create more complex categorical variables

Wrangling for probabilities

What is the probability that someone was admitted?

```
1 admissions |>
2   count(Decision) |>
3   mutate(prob = n/sum(n)) |>
4   select(-n)
```

A tibble: 2 × 2

	Decision	prob
	<chr>	<dbl>
1	Admit	0.388
2	Reject	0.612

What is the probability that someone was admitted, conditioned on gender?

```
1 admissions |>
2   count(Gender, Decision) |>
3   group_by(Gender) |>
4   mutate(cond_prob = n/sum(n)) |>
5   select(-n)
```

A tibble: 4 × 3

Groups: Gender [2]

	Gender	Decision	cond_prob
	<chr>	<chr>	<dbl>
1	Female	Admit	0.304
2	Female	Reject	0.696
3	Male	Admit	0.445
4	Male	Reject	0.555

- How might I extend to also condition on Department?

More complex categorical variables

Suppose I want to create a new variable called `Dept2` that takes the values:

- “Group 1” if someone applied to Department A or B
- “Group 2” if someone applied to Department C or D
- “Group 3” if someone applied to Department E or F

```
1 # option 1 (awful): nested if_else()
2 admissions |>
3   mutate(Dept2 = if_else(Dept %in% c("A", "B"), "Group 1",
4                           if_else(Dept %in% c("C", "D"), "Group 2",
5                                   "Group 3")))
5
```

```
# A tibble: 5 × 4
  Decision Gender Dept  Dept2
  <chr>      <chr> <chr> <chr>
1 Reject   Female C      Group 2
2 Admit    Male   A      Group 1
3 Reject   Female E      Group 3
4 Reject   Male   B      Group 1
5 Reject   Female C      Group 2
```

case_when()

We will use the `case_when()` function which generalizes `if_else()`. We use the following notation: `<logical condition> ~ <value of variable>`. Different “ifs” are separated by commas, and the logical conditions are checked sequentially.

```
1 admissions |>
2   mutate(Dept2 = case_when(
3     Dept %in% c("A", "B") ~ "Group 1",
4     Dept %in% c("C", "D") ~ "Group 2",
5     Dept %in% c("E", "F") ~ "Group 3",
6   ))
```

A tibble: 5 × 4

	Decision	Gender	Dept	Dept2
	<chr>	<chr>	<chr>	<chr>
1	Reject	Female	C	Group 2
2	Admit	Male	A	Group 1
3	Reject	Female	E	Group 3
4	Reject	Male	B	Group 1
5	Reject	Female	C	Group 2

```
1 # The following is also acceptable, but
2 # relies on sequential ordering:
3 admissions |>
4   mutate(Dept2 = case_when(
5     Dept %in% c("A", "B") ~ "Group 1",
6     Dept %in% c("C", "D") ~ "Group 2",
7     T ~ "Group 3",
8   )) |>
9   sample_frac()
```

A tibble: 5 × 4

	Decision	Gender	Dept	Dept2
	<chr>	<chr>	<chr>	<chr>
1	Reject	Female	C	Group 2
2	Admit	Male	A	Group 1
3	Reject	Female	E	Group 3
4	Reject	Male	B	Group 1
5	Reject	Female	C	Group 2

Prettier tables using `kable()`

- When we finish wrangling, the output is always a data frame
 - While this is so useful for coding, it's not the most beautiful when rendering!
 - How can we make turn the data frame into a beautiful table?
- We will need to first install the `kableExtra` library.

Currently:

```
1 admissions |>
2   count(Decision) |>
3   mutate(prob = n/sum(n))
```

```
# A tibble: 2 × 3
  Decision      n  prob
  <chr>    <int> <dbl>
1 Admit    1755 0.388
2 Reject   2771 0.612
```

- Using `kable()` (note we can specify number of digits)

```
1 library(kableExtra)
2 admissions |>
3   count(Decision) |>
4   mutate(prob = n/sum(n)) |>
5   kable(digits = 3)
```

Decision	n	prob
Admit	1755	0.388
Reject	2771	0.612